

BUET CSE 310 - Compiler Sessional

Lexical Analysis Practice Problems

Delta Modifications & Extra Features

July 2026

Introduction

This document contains practice problems designed for the CSE 310 Lexical Analysis online assignment. The problems are formulated as "delta tasks"—extensions to your existing base assignment. They are divided into two sections: Feature Additions and Error Handling Enhancements. Sample input and output logs are provided to help you verify your flex implementation.

1 Feature Additions

- 1. Hexadecimal and Octal Literals:** Extend your `CONST_INT` logic to support Hexadecimal (e.g., `0x1A2B`, `0Xff`) and Octal (e.g., `0777`).
 - Hex starts with `0x` or `0X` followed by `[0-9a-fA-F]+`.
 - Octal starts with `0` followed by `[0-7]+`.
 - *Token:* `<CONST_INT, lexeme>`
- 2. Bitwise Operators and Compound Assignments:**
 - Identify `&`, `|`, `^`, `~`, `<<`, `>>` as `BITOP`.
 - Identify `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=` as `COMPOUND_ASSIGN`.
 - *Hint:* Ensure `>>=` is not misidentified as `>>` followed by `=`.
- 3. Nested Multi-line Comments:** Modify your comment rules using Flex States (`%x`) to support nested multi-line comments.
 - *Example:* `/* outer /* inner */ outer */` should be treated as a single comment.
- 4. Python-style Digit Separators:** Modify `CONST_INT` and `CONST_FLOAT` to allow a single underscore between digits (e.g., `1_000_000`, `3.141_592`).
- 5. C++ Style Raw Strings:** Implement Raw Strings where escape sequences like `\n` are NOT converted.
 - *Task:* Recognize `R"( ... )"`.

- *Output:* <STRING, ...> containing the exact literal characters.
- 6. Preprocessor Directives (Macro Recognition):**
 - Detect `#define` and `#include` directives.
 - For `#include <filename.h>`, generate a token `<INCLUDE, filename.h>`.
 - 7. Hexadecimal Floating Point Constants:** Recognize C99 hex-floats using `p` or `P` for the exponent.
 - *Pattern:* `0x` + hex-digits + `.` + hex-digits + `[pP]` + `[+-]? + decimal-digits` (e.g., `0x1.fp3`).
 - 8. Binary Literals:** Support binary integers starting with `0b` or `0B` (e.g., `0b1011`).
 - 9. Character Literal Hex-Escapes:** Support `'\xHH'` where `HH` are two hex digits. You must convert `\x41` to `A` before printing to the token file.
 - 10. Type Qualifiers and sizeof:** Add `sizeof`, `const`, `volatile`, `struct`, `union`, `typedef` to your keywords table.

2 Error Handling Enhancements

- 11. Invalid Escape Sequences in Characters/Strings:** The current assignment catches "Empty character" and "Unterminated string". Enhance this by detecting invalid escape sequences (e.g., `'\q'`, `"hello\m"`).
 - *Error Message:* Invalid escape sequence `\q`
- 12. Identifier Length Limit Exceeded:** Some C compilers enforce an identifier length limit. Add a rule/action to track identifier length. If an identifier exceeds 31 characters, report an error instead of treating it as a valid ID.
 - *Error Message:* Identifier length exceeded
- 13. Unmatched Parentheses/Curly Braces Tracking:** Keep a counter for `(` and `{`. If a closing parenthesis `)` or curly brace `}` is encountered without a preceding open counterpart, report a lexical error immediately.
 - *Error Message:* Unmatched closing parenthesis or Unmatched closing brace
- 14. Invalid Suffix/Format on Extended Number Literals:** Expand your "Invalid suffix" error to catch malformed additions from Section 1.
 - `089` → Invalid Octal Number
 - `0x1G3` → Invalid Hexadecimal Number
 - `0b1021` → Invalid Binary Number
 - `100_` or `_100` → Invalid usage of digit separator

3 Sample Input and Output

This section provides a unified code block that tests several of the new features and errors.

Sample Input (input_prac.txt)

```
1 int main() {
2     // 1. Extended Literals
3     int h = 0x1A;
4     int b = 0b1011;
5
6     // 2. Extended Errors
7     int err1 = 089;
8     int err2 = 0x1GZ;
9
10    // 3. Operators & Separators
11    h += 1_000;
12    int i = >>; // Isolated bitop
13
14    // 4. Nested Comments
15    /* Outer /* Inner */ */
16
17    // 5. Hex Escapes & Invalid Escapes
18    char ch1 = '\x41';
19    char ch2 = '\q';
20
21    // 6. Length and Parenthesis check
22    int this_identifier_is_way_too_long_for_c = 5;
23    )
24
25    return 0;
26 }
```

Expected Output Log (input_prac_log.txt)

```
Line no 1: Token ;INT; Lexeme int found
Line no 1: Token ;ID; Lexeme main found
Line no 1: Token ;LPAREN; Lexeme ( found
Line no 1: Token ;RPAREN; Lexeme ) found
Line no 1: Token ;LCURL; Lexeme { found

Line no 2: Token ;COMMENT; Lexeme // 1. Extended Literals found

Line no 3: Token ;INT; Lexeme int found
Line no 3: Token ;ID; Lexeme h found
Line no 3: Token ;ASSIGNOP; Lexeme = found
Line no 3: Token ;CONST_INT; Lexeme 0x1A found
Line no 3: Token ;SEMICOLON; Lexeme ; found

Line no 4: Token ;INT; Lexeme int found
Line no 4: Token ;ID; Lexeme b found
Line no 4: Token ;ASSIGNOP; Lexeme = found
Line no 4: Token ;CONST_INT; Lexeme 0b1011 found
Line no 4: Token ;SEMICOLON; Lexeme ; found

Line no 6: Token ;COMMENT; Lexeme // 2. Extended Errors found
Line no 7: Token ;INT; Lexeme int found
Line no 7: Token ;ID; Lexeme err1 found
Line no 7: Token ;ASSIGNOP; Lexeme = found
Error at line no 7: Invalid Octal Number 089
Line no 7: Token ;SEMICOLON; Lexeme ; found

Line no 8: Token ;INT; Lexeme int found
Line no 8: Token ;ID; Lexeme err2 found
Line no 8: Token ;ASSIGNOP; Lexeme = found
Error at line no 8: Invalid Hexadecimal Number 0x1GZ
Line no 8: Token ;SEMICOLON; Lexeme ; found

Line no 10: Token ;COMMENT; Lexeme // 3. Operators & Separators found
Line no 11: Token ;ID; Lexeme h found
Line no 11: Token ;COMPOUND_ASSIGN; Lexeme += found
Line no 11: Token ;CONST_INT; Lexeme 1_000 found
Line no 11: Token ;SEMICOLON; Lexeme ; found

Line no 12: Token ;INT; Lexeme int found
Line no 12: Token ;ID; Lexeme i found
Line no 12: Token ;ASSIGNOP; Lexeme = found
Line no 12: Token ;BITOP; Lexeme ; found
Line no 12: Token ;SEMICOLON; Lexeme ; found

Line no 14: Token ;COMMENT; Lexeme // 4. Nested Comments found
Line no 15: Token ;COMMENT; Lexeme /* Outer /* Inner */ */ found

Line no 17: Token ;COMMENT; Lexeme // 5. Hex Escapes & Invalid Escapes found
Line no 18: Token ;CHAR; Lexeme char found
Line no 18: Token ;ID; Lexeme ch1 found
Line no 18: Token ;ASSIGNOP; Lexeme = found
Line no 18: Token ;CONST_CHAR; Lexeme '\x41' found -; ;CONST_CHAR, A;
Line no 18: Token ;SEMICOLON; Lexeme ; found

Line no 19: Token ;CHAR; Lexeme char found
Line no 19: Token ;ID; Lexeme ch2 found
Line no 19: Token ;ASSIGNOP; Lexeme = found
Error at line no 19: Invalid escape sequence \q
Line no 19: Token ;SEMICOLON; Lexeme ; found

Line no 21: Token ;COMMENT; Lexeme // 6. Length and Parenthesis check found
```